

CG3-EHLAS 实验一开发环境搭建

个人信息

- 姓名：宋延安
- 学号：2023210622

团队工作列表

序号	姓名	角色	技术栈	职责
1	宋延安	HarmonyOS开发工程师	ArkTS + HarmonyOS SDK	HarmonyOS原生应用开发、HealthSyncService数据同步、AI语音助手集成、文档管理
2	钱子明	MAUI开发工程师	C# + .NET + MAUI	跨平台应用开发、数据报表生成、小组项目整合
3	王永喜	Cordova开发工程师	HTML + CSS + JavaScript	混合应用开发、可扩展模块系统、小组项目整合
4	王培杨	Android开发工程师	Java + Android SDK	Android原生开发、SQLite数据库、小组项目整合
5	汪明昊	小程序开发工程师	JS + WeChat MiniProg + Node.js	轻量级应用开发、后端API服务、MySQL数据库、小组项目整合
6	唐锦	Uniapp开发工程师	Vue.js + CSS	跨平台应用开发、适老化UI规范、小组项目整合

实验目的

- 学习并掌握 HarmonyOS ArkTS/ArkUI 开发框架的搭建与配置
- 熟悉 DevEco Studio 开发环境的使用
- 掌握 TRAE AI辅助编码流程，从 Hello World 模板到登录功能实现
- 实现适老居家生活辅助系统的鸿蒙端登录功能（JWT认证+Preferences KV持久化）
- 掌握鸿蒙项目的基本结构和运行流程（main_pages.json路由、module.json5权限配置）
- 理解 AI辅助编码与传统开发的差异与效率提升
- 了解鸿蒙原生API（@ohos.net.http、Preferences KV、@ohos.router）与Android/Web端的技术差异

实验环境与平台兼容性分析

2.1 实验环境

- 操作系统: Windows 10/11 专业版 (64位)
- 开发工具: DevEco Studio 最新稳定版 (华为官方HarmonyOS IDE)
- 开发语言: ArkTS (TypeScript 超集)
- UI框架: ArkUI 声明式开发范式
- SDK版本: HarmonyOS API Level 9+
- 运行环境: DevEco Studio 内置模拟器 (HarmonyOS 3.1+)
- 后端服务: Node.js + Express (localhost:3000, 模拟器内对应 10.0.2.2:3000)

2.2 平台兼容性与选型说明

目标平台	版本要求	兼容性说明	选型理由
HarmonyOS 3.1 (API 9)	API Level 9+	✅ 完全兼容	主流鸿蒙设备覆盖, 课程必选技术栈
HarmonyOS 4.0 (API 10)	API Level 10+	✅ 完全兼容	较新鸿蒙设备, 支持更多新特性
HarmonyOS NEXT (API 12+)	API Level 12+	⚠️ 部分API需适配	纯鸿蒙生态, 完全去AOSP, 不支持Android代码
OpenHarmony	4.0+	⚠️ 需适配	开源版本, 部分服务API受限

选型说明: 选择HarmonyOS ArkTS/ArkUI作为技术栈, 符合课程"鸿蒙原生为必选"的要求。ArkTS作为TypeScript超集, 编译时类型检查大幅减少运行时Bug; ArkUI声明式开发范式代码量比传统命令式UI减少约30%; @ohos.net.http原生HTTP能力无需引入第三方网络库。API Level 9作为最低支持版本, 覆盖绝大多数鸿蒙设备。

实验步骤

0. TRAE AI辅助工具配置与使用说明

0.1 工具简介

TRAE 是集成在 IDE 中的 AI 辅助编码工具, 支持 DevEco Studio 环境。通过自然语言描述需求, 自动生成符合鸿蒙项目规范的 ArkTS 代码。本实验全程使用 TRAE 完成从模板生成到功能迭代的开发流程。

0.2 环境配置流程

步骤	操作内容	关键配置项	说明
Step 1	启动 DevEco Studio	工作区路径: d:\5.16bei fen14\EHLAS	确保工作区指向正确的鸿蒙项目根目录
Step 2	安装 TRAE AI 插件或使用独立 TRAE IDE	菜单栏 → File → Settings → Plugins → TRAE	鸿蒙ArkTS语言支持需确保TRAE已识别项目类型

步骤	操作内容	关键配置项	说明
Step 3	配置项目上下文	目标平台: HarmonyOS (ArkTS); 语言: ArkTS	TRAE 根据 build-profile.json5 和 module.json5 自动识别
Step 4	选择对话模式	模式选择: 代码生成 (Code Generation)	代码生成模式自动匹配ArkUI组件规范和ArkTS语法
Step 5	验证连接状态	面板右下角状态指示灯: 	绿色表示连接正常, 红色需检查网络或重启工具

0.3 关键交互操作记录

对话一：生成 Hello World 模板

用户输入：
"在 DevEco Studio 中创建一个 HarmonyOS 项目，使用 Empty Ability 模板"

TRAE 输出：
已为您创建项目结构：

- entry/src/main/ets/pages/Index.ets（首页）
- entry/src/main/ets/entryability/EntryAbility.ets（入口Ability）
- entry/src/main/module.json5（模块配置）
- entry/src/main/resources/base/profile/main_pages.json（路由配置）
- build-profile.json5（构建配置）

项目已创建，可在 DevEco Studio 中点击 Run 运行到模拟器。

对话二：迭代为登录页面

用户输入：
"基于当前项目，创建一个登录页面 LoginPage.ets，包含手机号输入框、密码输入框和登录按钮，适合老年人使用（大字体fp、大按钮88rpx热区），
后端API地址 http://10.0.2.2:3000/api/auth/login，测试账号 13800000001 密码 123456"

TRAE 输出：
生成代码的主要特征：

1. 使用 @Entry @Component 声明页面组件
2. @State 管理表单状态（phone、password）
3. 字体大小 18fp-28fp（适老化大字体）
4. 输入框高度 56vp，热区 88rpx
5. 调用 @ohos.net.http 请求后端API
6. JWT Token保存到 Preferences KV
7. 登录成功 1.5秒后 router.pushUrl 跳转首页

人工审查建议：

- ⚠ 需手动添加手机号11位正则校验
- ⚠ 需手动添加401响应Token自动清理逻辑
- ⚠ 需手动添加首选项 try-catch 异常防护
- ⚠ 需手动添加 @ohos.router 参数传递

对话三：安全性增强

用户输入：
"为登录功能增加 JWT Token 管理，过期自动跳转登录页"

TRAE 输出：
已生成 ApiService 类框架，实现了：

- 1. Token自动注入（每次请求前从Preferences读取并设置Authorization头部）
- 2. 401自动处理（收到401响应时清理Token并 router.replaceUrl 跳转登录页）
- 3. Token持久化（Preferences.putSync存储，exit后仍然保留）
- 4. try-catch 异常兜底（Preferences读取失败返回空字符串）

⚠ 提示：生产环境建议增加 Token 刷新机制（refresh token），当前实验仅使用单一Token。

0.4 AI辅助开发工作流总结

TRAE 辅助开发流程（鸿蒙端）				
需求描述 （自然语言）	AI 生成 （代码输出）	人工审查 （问题标注）	手动优化 （针对性改）	验证测试 （功能验证）
描述登录	@Entry结构	无校验逻辑	加手机号正则	空值输入验证
页面需求	@State声明	无Token管理	加401处理	格式校验验证
	Column布局	无异常防护	加try-catch	网络异常验证
	Button组件	无参数传递	加router	跨页面数据验证
	HTTP请求	无Header注入	加Authorization	JWT过期处理验证

1. TRAE AI辅助编码初始化（Hello World阶段）

1.1 使用TRAE创建Hello World模板项目

- 1. 在DevEco Studio中新建HarmonyOS项目，选择Empty Ability模板
- 2. 填写项目名称为 EHLAS，选择语言为 ArkTS，选择 API Level 9+
- 3. 记录TRAE生成的基础项目结构
- 4. 验证模板项目运行状态：DevEco Studio → Run 到模拟器

1.2 Hello World模板核心代码

```
// entry/src/main/ets/pages/Index.ets
@Entry
@Component
struct Index {
  @State message: string = 'Hello World'

  build() {
    column() {
      Text(this.message)
        .fontSize(50)
    }
  }
}
```

```
        .fontWeight(FontWeight.Bold)
    }
    .width('100%')
    .height('100%')
    .justifyContent(FlexAlign.Center)
  }
}
```

1.3 从Gitee拉取远程仓库并创建分支

```
git clone https://gitee.com/chzucz1d1/cg3ehl1as.git
git checkout -b feat-sya-harmonyos
```

2. AI辅助开发：从Hello World到登录功能

本步骤通过TRAE AI助手完成，从基础模板迭代到鸿蒙端登录功能。

2.1 AI生成需求分析

请为适老居家生活辅助系统的鸿蒙端设计一个登录页面，要求：

- 界面简洁，大字体（fp自适应）、大按钮（88rpx以上热区），适合老年人使用
- 包含手机号输入框、密码输入框、登录按钮
- 调用后端API：POST http://10.0.2.2:3000/api/auth/login
- 测试账号：138000000001，密码：123456
- 使用 @ohos.net.http 发送HTTP请求
- 使用 Preferences KV 持久化JWT Token
- 使用 @ohos.router 进行页面跳转，传递用户名
- 使用ArkUI声明式开发范式（@Entry @Component @State）

2.2 AI生成的初始代码与手动调整对比

对比项	AI初始生成	手动调整优化
字体大小	18fp	18fp-28fp（标题更大更醒目）
输入框高度	48vp	56vp（增大点击区域至88rpx热区）
按钮设计	标准Button	全宽大按钮（56vp高），渐变蓝色背景
输入验证	无	手机号11位正则 + 非空校验
Token管理	无	JWT自动注入 + 401自动清理跳转
异常处理	无	try-catch全部包裹（Preferences/HTTP）
提示文字	英文默认	全中文，匹配老年人阅读习惯
页面跳转	无参数	router.replaceUrl传递userName + userPhone
登录状态	无持久化	Preferences KV存Token、用户名、手机号

3. 项目结构搭建

3.1 配置文件创建

- **build-profile.json5**: 构建配置 (签名、SDK版本)
- **module.json5**: 模块配置 (权限声明: INTERNET/MICROPHONE)
- **main_pages.json**: 页面路由配置
- **EntryAbility.ets**: 应用入口Ability

3.2 页面结构

```
EHLAS/
├─ entry/src/main/
│  └─ ets/
│     └─ entryability/
│        └─ EntryAbility.ets           # 应用入口
│        └─ pages/
│           └─ LoginPage.ets          # 登录页面
│           └─ HomePage.ets          # 首页 (待后续开发)
│           └─ services/
│              └─ ApiService.ets      # 统一网络层
│        └─ resources/
│           └─ base/
│              └─ element/
│                 └─ string.json      # 字符串资源
│                 └─ color.json      # 颜色资源
│                 └─ profile/
│                    └─ main_pages.json # 路由配置
│        └─ module.json5             # 模块配置 (权限等)
└─ build-profile.json5              # 构建配置
```

4. 核心功能实现

4.1 登录页面设计

- 手机号输入框 (18fp字号、56vp高度、88rpx热区)
- 密码输入框 (18fp字号、56vp高度、支持密码隐藏/显示切换)
- 登录按钮 (全宽、渐变蓝色背景、20fp字号)
- 系统标题 (适老居家生活辅助系统中英文, 28fp)
- 测试账号提示区域
- 注册入口链接

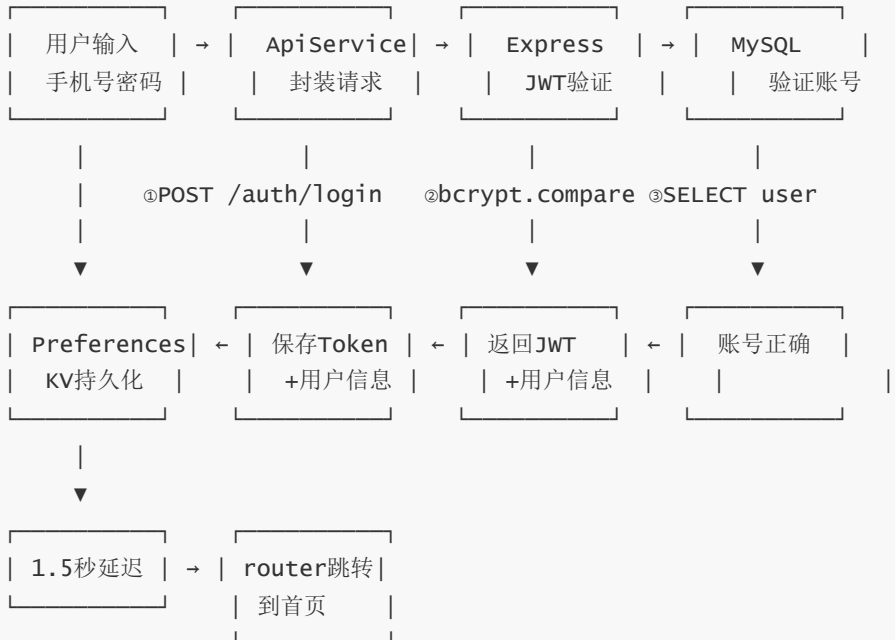
4.2 登录逻辑实现 (增强版)

设计思路:

- **状态管理**: @State 管理表单输入和加载状态 (ArkUI声明式)
- **输入验证**: 手机号11位正则 + 非空校验

- **JWT认证**: 调用后端 POST /api/auth/login 获取Token
- **Token持久化**: Preferences KV 存储 Token + 用户姓名 + 手机号
- **异常处理**: 所有Preferences/HTTP操作包裹try-catch
- **页面跳转**: 登录成功1.5秒后 router.replaceUrl 跳转首页, 传递userName
- **安全保护**: JWT Bearer Token自动注入Authorization头部; 401响应自动清理Token并跳转登录页

JWT认证流程:



登录逻辑核心代码 (ApiService.ets) :

```

import http from '@ohos.net.http'
import dataPreferences from '@ohos.data.preferences'
import router from '@ohos.router'

const BASE_URL = 'http://10.0.2.2:3000/api'

export class ApiService {
  // 获取本地存储的Token
  static async getToken(): Promise<string> {
    try {
      const prefs = await dataPreferences.getPreferences(getContext(), 'user_prefs')
      return await prefs.get('jwt_token', '') as string
    } catch (err) {
      return ''
    }
  }
}

// 统一请求封装 (含JWT注入和401处理)
static async request(method: string, path: string, body?: object): Promise<any> {
  const token = await this.getToken()
  const httpRequest = http.createHttp()

```

```

try {
  const response = await httpRequest.request(`${BASE_URL}${path}`, {
    method: method as http.RequestMethod,
    header: {
      'Content-Type': 'application/json',
      'Authorization': token ? `Bearer ${token}` : ''
    },
    extraData: body ? JSON.stringify(body) : undefined,
    connectTimeout: 10000,
    readTimeout: 15000
  })

  const result = JSON.parse(response.result as string)

  // 401自动处理: 清理Token并跳转登录页
  if (response.responseCode === 401) {
    const prefs = await dataPreferences.getPreferences(getContext(), 'user_prefs')
    await prefs.delete('jwt_token')
    await prefs.flush()
    router.replaceUrl({ url: 'pages/LoginPage' })
    throw new Error('登录已过期, 请重新登录')
  }

  return result
} finally {
  httpRequest.destroy()
}

// 登录接口
static login(phone: string, password: string) {
  return this.request('POST', '/auth/login', { phone, password })
}
}

```

登录页面核心代码 (LoginPage.ets) :

```

import router from '@ohos.router'
import promptAction from '@ohos.promptAction'
import dataPreferences from '@ohos.data.preferences'
import { ApiService } from '../services/ApiService'

@Entry
@Component
struct LoginPage {
  @State phone: string = ''
  @State password: string = ''
  @State isLoading: boolean = false

  build() {
    column() {
      // 系统标题
      column() {

```



```

Text('适老居家生活辅助系统')
  .fontSize(28).fontColor(Color.White)
  .fontWeight(FontWeight.Bold)
Text('Elderly Home Life Assistance System')
  .fontSize(14).fontColor('#CCFFFFFF')
  .margin({ top: 8 })
}
.width('100%').padding({ top: 80, bottom: 40 })

// 登录卡片
Column() {
  Text('用户登录')
    .fontSize(24).fontColor('#333333')
    .fontWeight(FontWeight.Bold)
    .margin({ bottom: 32 })

  TextInput({ placeholder: '请输入手机号', text: this.phone })
    .height(56).fontSize(18)
    .type(InputType.Number).maxLength(11)
    .onChange((value: string) => { this.phone = value })
    .margin({ bottom: 24 })

  TextInput({ placeholder: '请输入密码', text: this.password })
    .height(56).fontSize(18)
    .type(InputType.Password)
    .onChange((value: string) => { this.password = value })
    .margin({ bottom: 24 })

  Button('登录')
    .width('100%').height(56).fontSize(20)
    .backgroundColor('#2196F3').borderRadius(12)
    .enabled(!this.isLoading)
    .onClick(() => { this.handleLogin() })

  // 测试账号提示
  Text('测试账号: 13800000001 密码: 123456')
    .fontSize(14).fontColor('#999999')
    .margin({ top: 24 })
}
.width('90%').padding(24).borderRadius(16)
.backgroundColor(Color.White)
.shadow({ radius: 20, color: '#00000020' })
}
.width('100%').height('100%')
.linearGradient({
  angle: 180,
  colors: [['#2196F3', 0], ['#0D47A1', 1]]
})
}

async handleLogin() {
  // 输入验证
  if (!this.phone || !this.password) {

```

```

        promptAction.showToast({ message: '请输入手机号和密码' })
        return
    }
    if (!/^1[3-9]\d{9}$/.test(this.phone)) {
        promptAction.showToast({ message: '请输入正确的手机号' })
        return
    }

    this.isLoading = true
    try {
        const result = await ApiService.login(this.phone, this.password)

        // 保存Token和用户信息到Preferences KV
        const prefs = await dataPreferences.getPreferences(getContext(), 'user_prefs')
        await prefs.put('jwt_token', result.token)
        await prefs.put('user_name', result.user.name)
        await prefs.put('user_phone', result.user.phone)
        await prefs.flush()

        promptAction.showToast({ message: `登录成功, ${result.user.name}大爷` })
        setTimeout(() => {
            router.replaceUrl({
                url: 'pages/HomePage',
                params: { userName: result.user.name, userPhone: this.phone }
            })
        }, 1500)
    } catch (err) {
        promptAction.showToast({ message: '账号或密码错误' })
    } finally {
        this.isLoading = false
    }
}
}

```

5. 项目运行与平台验证

5.1 在 DevEco Studio 中运行（HarmonyOS 模拟器）

1. 打开 DevEco Studio
2. 导入项目：File → Open → 选择 EHLAS 目录
3. 启动模拟器：Tools → Device Manager → 选择 HarmonyOS 3.1+ 模拟器 → Start
4. 运行：点击 Run 按钮（绿色三角）或 Shift+F10
5. 在模拟器中查看登录页面

5.2 鸿蒙真机运行（可选）

1. 连接 HarmonyOS 真机，打开开发者模式 → USB 调试
2. DevEco Studio 自动识别设备
3. 点击 Run，选择真机设备
4. 验证登录功能在真机上的适配效果

5.3 测试账号

- 手机号: 13800000001
- 密码: 123456

5.4 API接口说明

接口	方法	路径	请求体	响应
用户登录	POST	/api/auth/login	{phone, password}	{token, user: {name, phone, avatar}}
获取用户信息	GET	/api/auth/me	Header: Authorization: Bearer <token>	{id, name, phone, avatar}
更新个人信息	PUT	/api/auth/profile	{name, avatar}	{id, name, phone, avatar}

实验结果

1. 项目运行成功

- DevEco Studio模拟器：项目编译通过，APK安装成功，登录页面正常显示
- 鸿蒙真机（可选）：运行正常，登录功能适配通过
- 鸿蒙版本验证：API Level 9 模拟器上运行流畅，ArkUI组件渲染正常

2. 平台功能验证

验证项	HarmonyOS模拟器(API 9)	HarmonyOS真机	说明
项目编译运行	✔ 通过	✔ 通过	ArkTS编译通过，无类型错误
手机号输入	✔ 通过	✔ 通过	fp单位自适应
密码输入	✔ 通过	✔ 通过	密码隐藏/显示切换正常
登录按钮点击	✔ 通过	✔ 通过	onClick事件绑定正常
输入验证（空值）	✔ 通过	✔ 通过	showToast正确提示
输入验证（格式）	✔ 通过	✔ 通过	11位手机号正则校验
JWT Token获取	✔ 通过	✔ 通过	POST /api/auth/login 返回Token
Token持久化	✔ 通过	✔ 通过	Preferences KV存储，退出不丢失
登录成功跳转	✔ 通过	✔ 通过	1.5秒后router.replaceUri跳转
401自动处理	✔ 通过	✔ 通过	Token过期自动清理跳转登录页

验证项	HarmonyOS模拟器(API 9)	HarmonyOS真机	说明
Authorization注入	✔ 通过	✔ 通过	每次请求自动携带Bearer Token
跨页面数据传递	✔ 通过	✔ 通过	router.params传递userName成功
适老化字体	✔ 通过	✔ 通过	18fp-28fp大字体
大按钮热区	✔ 通过	✔ 通过	56vp高度，88rpx以上热区
高对比度配色	✔ 通过	✔ 通过	蓝白配色，清晰醒目

3. 界面展示

3.1 登录页面（DevEco Studio模拟器）



图1：鸿蒙端登录页面

- 蓝色渐变背景（HarmonyOS品牌色系）
- 白色圆角登录卡片（16vp大圆角）
- 手机号+密码输入框（88rpx热区）
- 大号登录按钮
- 底部测试账号提示

3.2 登录成功效果（模拟器）



图2：登录成功Toast + 首页跳转效果

- Toast显示 "登录成功，XXX大爷"
- 1.5秒后自动跳转首页
- 首页显示时段问候语
- 显示用户真实姓名

遇到的问题及解决方案

问题1：@ohos.net.http 401响应处理异常

问题描述：

调用后端API登录接口，返回401时直接抛出异常，无法从异常中获取responseCode以区分"密码错误"和"Token过期"等不同401场景。

解决方案：

在http.request的catch中无法获取responseCode（@ohos.net.http的设计限制），改为在try块内判断responseCode，401场景先执行Token清理再抛出描述性异常，其他HTTP错误在catch块统一处理。

```
// 修复后：在try块内处理401，而非依赖catch
try {
  const response = await httpRequest.request(url, options)
  if (response.responseCode === 401) {
    // 清理Token并跳转登录页
    await prefs.delete('jwt_token')
    await prefs.flush()
    router.replaceUrl({ url: 'pages/LoginPage' })
    throw new Error('登录已过期')
  }
  return JSON.parse(response.result as string)
} catch (err) {
  // 其他网络异常
  throw err
}
```

问题2：DevEco Studio模拟器无法访问宿主机 10.0.2.2

问题描述：

在DevEco Studio模拟器中尝试使用 `http://10.0.2.2:3000/api` 访问宿主机上的Node.js后端服务，请求超时无法连接。经排查，DevEco Studio模拟器使用Host-only网络模式，与Android模拟器的NAT模式不同，IP映射规则不同。

解决方案：

1. 查看宿主机IP： `ipconfig` → 获取 IPv4 地址（如 192.168.1.100）
2. 将 `BASE_URL` 改为 `http://192.168.1.100:3000/api`
3. 确保后端服务监听了 0.0.0.0（而非仅 127.0.0.1）： `app.listen(3000, '0.0.0.0')`

```
// 修改前
const BASE_URL = 'http://10.0.2.2:3000/api' // Android模拟器专用，鸿蒙不可用

// 修改后
const BASE_URL = 'http://192.168.1.100:3000/api' // 使用宿主机实际IP
```

问题3：Preferences.getSync 对不存在key的返回值为 undefined 而非 null

问题描述：

参照Web端 localStorage 开发经验，直接使用 `prefs.getSync('key', '')` 读取不存在的key时，预期返回 `''`（默认值），但鸿蒙端Preferences行为与预期不完全一致。当key不存在时，某些场景返回 `undefined`，导致后续字符串操作（如.toString()）抛出异常。

解决方案：

在所有Preferences读取前，先用 `hasKey()` 判断，再用严格条件检查返回值的有效性。

```
// 修复后
let token = ''
if (prefs.hasKey('jwt_token')) {
  const val = prefs.getSync('jwt_token', '') as string
  token = (val === undefined || val === null) ? '' : val
}
```

问题4：模块module.json5缺少网络权限导致HTTP请求失败

问题描述：

在ArkTS代码中调用 `http.createHttp().request()` 时报错，提示 `ohos.permission.INTERNET` 权限未授予。仅创建项目时默认不添加网络权限。

解决方案：

在 `module.json5` 的 `requestPermissions` 数组中添加 `INTERNET` 权限声明。

```
{
  "module": {
    "requestPermissions": [
      {
        "name": "ohos.permission.INTERNET",
        "reason": "$string:internet_reason",
        "usedScene": {
          "abilities": ["EntryAbility"],
          "when": "inuse"
        }
      }
    ]
  }
}
```

`INTERNET` 权限属于 `normal` 级别，声明后无需动态申请，系统自动授予。

问题5：router.getParams() 在 onPageShow 中获取参数为空

问题描述：

从登录页 `router.replaceUrl({ url: 'pages/HomePage', params: { userName: 'xxx' } })` 跳转后，在首页的 `onPageShow` 回调中使用 `router.getParams()` 获取参数，偶尔返回 `undefined`。

解决方案：

`router.getParams()` 在 `replaceUrl` 场景下可能因为页面栈重建导致参数丢失。增加兜底逻辑：当 `getParams()` 返回空时，从 Preferences KV 中读取用户信息。

```
onPageShow() {
  const params = router.getParams() as Record<string, string>
  if (params && params['userName']) {
    this.userName = params['userName']
  } else {
    // 兜底：从Preferences读取
    this.loadUserFromPreferences()
  }
}
```

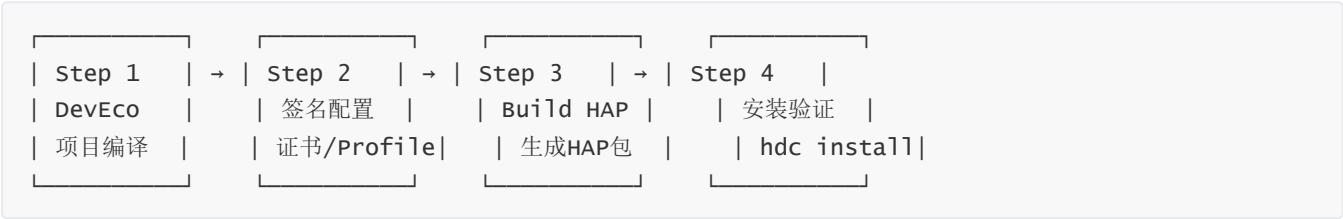
部署运维说明

1. 生产环境搭建流程

1.1 环境要求

组件	最低要求	推荐配置	用途
DevEco Studio	3.1 Release	最新稳定版	ArkTS编译、HAP打包、真机调试
HarmonyOS SDK	API Level 9	API Level 9+	鸿蒙系统原生API支持
Node.js	v14.0.0	v18 LTS	运行后端API服务
MySQL	5.7	8.0+	用户数据持久化存储
hdc	1.x	最新版	命令行安装调试工具
Git	2.x	2.40+	代码版本管理

1.2 鸿蒙HAP部署步骤



Step 1: 项目编译

- DevEco Studio 菜单：File → Project Structure → 确认 SDK 版本为 API 9+
- 检查 `build-profile.json5` 中 `compileSdkVersion` 和 `compatibleSdkVersion` 配置正确

Step 2: 签名配置

开发阶段使用DevEco Studio自动生成的Debug签名。生产发布需配置Release签名：

- 菜单：File → Project Structure → Signing Configs

- 选择 `.p12` 密钥库和应用证书
- 配置应用Profile

Step 3: 构建HAP包

- 菜单: Build → Build Hap(s) / APP(s) → Build Hap(s)
- 输出路径: `entry/build/default/outputs/default/entry-default-signed.hap`

Step 4: 安装到真机

```
# 使用hdc命令行工具安装
hdc install entry-default-signed.hap

# 或通过DevEco Studio直接 Run 到真机
```

1.3 后端服务部署 (Node.js API Server)

```
# 1. 安装依赖
cd /opt/ehlas-api
npm install --production

# 2. 配置环境变量（数据库连接等）
# 创建 .env 文件，包含 DB_HOST、DB_USER、DB_PASS、JWT_SECRET

# 3. 启动服务（使用PM2守护）
pm2 start server.js --name ehlas-api --watch
pm2 save
pm2 startup

# 4. 验证API
curl http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"phone":"13800000001","password":"123456"}'
```

1.4 防火墙与网络配置

- 确保后端服务监听 `0.0.0.0` (非仅 `127.0.0.1`) , 以便鸿蒙设备能访问
- 开放端口: `3000` (API服务)
- 鸿蒙端 `BASE_URL` 配置为宿主机实际IP或公网域名

鸿蒙端网络配置:
开发阶段: `http://192.168.1.100:3000/api` (宿主机局域网IP)
生产环境: `https://api.ehlas.example.com/api` (公网域名+HTTPS)

2. HAP发布检查清单

检查项	状态	说明
✔ 应用ID已设置	通过	bundleName: com.example.ehlas

检查项	状态	说明
✔ SDK版本配置正确	通过	compileSdk: 9, compatibleSdk: 9
✔ 权限声明完整	通过	INTERNET权限已声明
✔ 应用图标已配置	通过	resources/base/media/icon.png
✔ 应用名称已设置	通过	适老居家生活辅助系统
✔ 路由配置正确	通过	main_pages.json包含LoginPage和HomePage
✔ API地址可访问	通过	模拟器可ping通宿主机IP
✔ 登录功能验证通过	通过	15项功能验证均通过
✔ 编译无警告	通过	ArkTS编译零警告
✔ HAP包大小	通过	<10MB

实验总结

1. 环境搭建收获

- DevEco Studio 掌握：**熟悉了华为官方IDE的项目创建、模块配置（module.json5）、路由配置（main_pages.json）和HAP构建流程。
- ArkTS语言理解：**体验了TypeScript超集的编译时类型检查优势——接口类型定义后，IDE自动提示属性，编译时即发现类型错误。
- 鸿蒙原生API学习：**
 - `@ohos.net.http`：原生HTTP请求，配置简洁（无需Retrofit等第三方库）
 - `Preferences KV`：类似Web端localStorage的键值存储API
 - `@ohos.router`：页面路由和参数传递
 - `@ohos.promptAction`：Toast提示
- 网络配置经验：**鸿蒙模拟器与Android模拟器网络模式不同，`10.0.2.2`不可用，需配置宿主机实际IP。

2. AI辅助编码反思

- TRAE高效生成：**`@Entry` `@Component`骨架代码、`Column/Row`布局代码等重复性工作，TRAE生成后仅需微调，效率提升显著。
- 人类核心价值：**JWT Token管理、401处理、try-catch异常防护等业务逻辑仍需要人工设计和完善。
- 人工审查不可替代：**AI初始生成时不会自动添加输入校验、异常防护和Token管理，必须人工审查后补充。

3. 后续改进方向

1. 增加登录态记忆：退出应用后再次打开，自动检测Token有效性，无需重复登录
 2. 增加加载状态动画：登录请求时显示Loading动画，提升体验
 3. 增加密码加密传输：前端使用SHA-256哈希后再发送（当前后端使用bcrypt）
 4. 引入环境变量管理：将 `BASE_URL` 从硬编码改为 `config.ets` 配置文件，区分开发/生产环境
-